PRACTICAL - 01

Aim: Vending machine (Coin.java + Driver.java)

Code Screenshot:

Coin.java

```java
public enum Coin {
    ONE(1), TWO(2), FIVE(5), TEN(10);

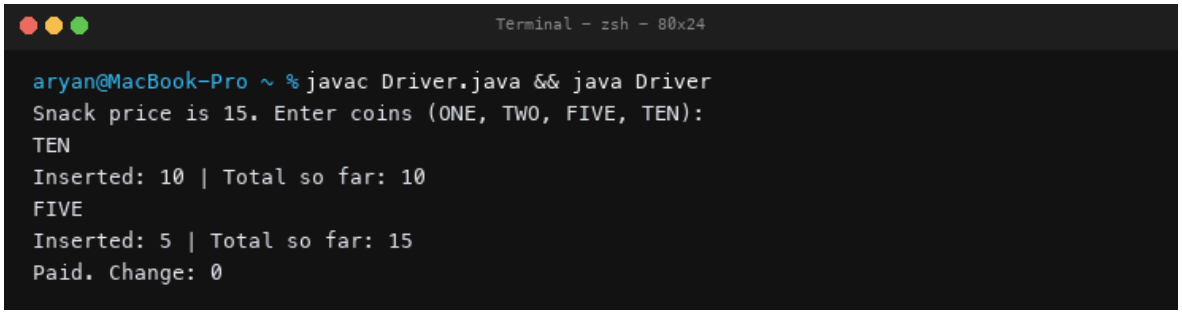
    private final int value;
    Coin(int value) { this.value = value; }
    public int getValue() { return value; }
}
```

Driver.java

```java
import java.util.Scanner;

public class Driver {
    public static void main(String[] args) {
        final int snackPrice = 15;
        int total = 0;
        Scanner sc = new Scanner(System.in);
        System.out.println("Snack price is 15. Enter coins (ONE, TWO, FIVE, TEN):");
        while (total < snackPrice) {
            String input = sc.next().toUpperCase();
            Coin coin = Coin.valueOf(input);
            int value = switch (coin) {
                case ONE -> 1;
                case TWO -> 2;
                case FIVE -> 5;
                case TEN -> 10;
            };
            total += value;
            System.out.println("Inserted: " + value + " | Total so far: " + total);
        }
        int change = total - snackPrice;
        System.out.println("Paid. Change: " + change);
        sc.close();
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned how to define and utilize Java enum types for fixed sets of constants.
- Understood modern switch expressions with arrow syntax and return values.
- Implemented accumulator logic for iterative coin transaction management.
- Handled console input parsing and enum constant resolution via valueOf().
- Computed exact monetary change based on user input.

Aim: Toll booth (Vehicle.java + Driver.java)

Code Screenshot:

Vehicle.java

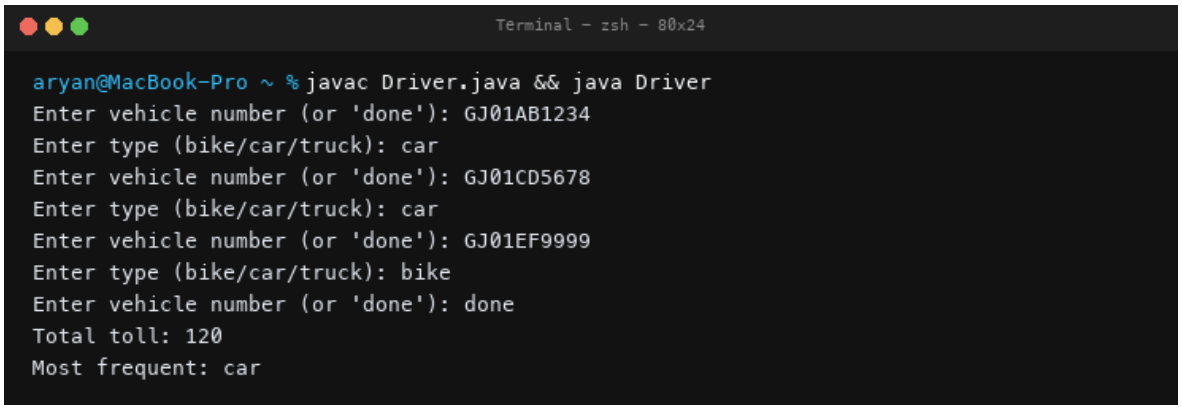
```java
public record Vehicle(String number, String type) {
    public Vehicle {
        if (number == null || type == null) {
            throw new IllegalArgumentException("Fields cannot be null");
        }
    }
}
```

Driver.java

```java
import java.util.Scanner;

public class Driver {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int totalToll = 0;
        int bikeCount = 0, carCount = 0, truckCount = 0;
        while (true) {
            System.out.print("Enter vehicle number (or 'done'):");
            String num = sc.next();
            if (num.equalsIgnoreCase("done")) break;
            System.out.print("Enter type (bike/car/truck):");
            String type = sc.next().toLowerCase();
            Vehicle v = new Vehicle(num, type);
            int toll = switch (v.type()) {
                case "bike" -> { bikeCount++; yield 20; }
                case "car" -> { carCount++; yield 50; }
                case "truck" -> { truckCount++; yield 150; }
                default -> 0;
            };
            totalToll += toll;
        }
        System.out.println("Total toll: " + totalToll);
        String frequent = (bikeCount >= carCount && bikeCount >= truckCount) ? "bike" :
                          (carCount >= truckCount ? "car" : "truck");
        System.out.println("Most frequent: " + frequent);
        sc.close();
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered Java record types for concise, immutable data holding.
- Used switch expressions with multi-statement blocks and the yield statement.
- Implemented frequency tracking counters across heterogeneous categories.
- Applied loop control flows with sentinel termination conditions.
- Gained hands-on experience in structured object-oriented problem solving.